

infotainer vk

Algorithms

→ An algorithm is a finite sequence of well defined, unambiguous instructions that takes some input, process it through a series of steps, and produces the required output to solve a specific problem.

Characteristics of an algorithm:

- ① Finiteness → termination after finite no. of steps
- ① Definiteness → No ambiguity (clear meaning)
- ① Input → zero or more
- ① Output → at least one output as result of execution.
- ① Effectiveness → each operation must be basic and executable in finite time
- ① Generality → algorithm should solve all valid instances of the problem, not just a single case.

Building blocks of an algorithm:

- ① Sequence → steps one after another in a fixed order
- ① Selection/decision → A condition decides which path to follow (e.g. if-else, switch)
- ① Iteration → A set of steps is repeated until a condition is met (e.g. for, while)
- ① Input
- ① Output

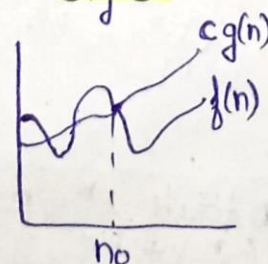
Judging efficiency of an algorithm

- ① Time complexity
- ② Space complexity
- ③ Scalability.

Algorithm analysis →

- ① Experimental
- ① Absolute - only on algo. asymptotic notations

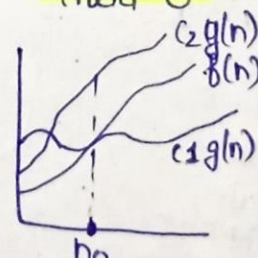
Big 'O'



$$f(n) = O(g(n))$$

- ① upper bound
- ① worst case time described (ज्यादा से ज्यादा)

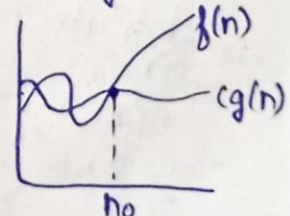
Theta 'Θ'



$$f(n) = \Theta(g(n))$$

- ① tight bound
- ① exact/same
- ① give both upper & lower bound

Omega 'Ω'



$$f(n) = \Omega(g(n))$$

- ① lower bound
- ① describe best case time complexity.
- ① (कम से कम)

Problem Solving steps

- ① Problem definition
- ② Algorithm design
- ③ Algorithm analysis
- ④ Implementation
- ⑤ Testing & debugging
- ⑥ Maintenance

Depth first search

Breadth first search

Algorithm complexity

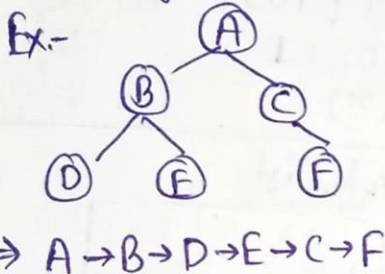
It refers to analysis of an algorithm performance in terms of time and space requirement as a function of input size.

Time complexity → amount of time an algo. takes to run as function of input size n .

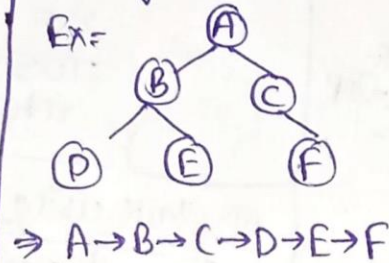
ex:- $O(1)$ - constant time
 $O(n)$ - linear
 $O(n^2)$ - quadratic

Space complexity → amount of memory required by an algorithm

- ① go deep first
- ① uses STACK
- ① concept → LIFO
- ① Travel 1 path fully
- ① Less memory
- ① work well on deep graphs
- ① Shortest path (not) guarantee



- ① go level-wise
- ① uses Queue
- ① concept → FIFO
- ① explore all neighbours 1st
- ① More memory
- ① work well on wide graphs.
- ① Shortest path guarantee



Time complexity

Space complexity

- | Time complexity | Space complexity |
|--|--|
| ① time taken by algo. to execute | ① Memory needed by algo. |
| ① Time counted for all statement | ① Memory counted for all variables, input & output |
| ① primary determinant → size of input data | ① primary determinant → variable size |
| ① concern → speed | ① concern → memory usage |
| ① Measured → No. of operation in | ① Measured → memory per variable in |
| ① goal → reduce execution time | ① goal → minimize memory usage |

Sparse matrix → matrix with most zero element

Ex:- $A = \begin{bmatrix} 0 & 0 & 5 \\ 0 & 9 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

Storage method = ① Tuple representation
= ② Compressed sparse row reprs.

Ex:- for i in range(1, n+1):
 print(i)

loop run n time.

Time complexity = $O(n)$

Ex:- storing n no. in array.
 $arr = [0] * n$
algo allocates array of size n
Space complexity = $O(n)$

Application of DFS:-

- ① cycle detection
- ② topological sorting
- ③ find connected components
- ④ maze solving

Application of BFS:-

- ① shortest path finding
- ② web crawling
- ③ tree traversal
- ④ Image processing

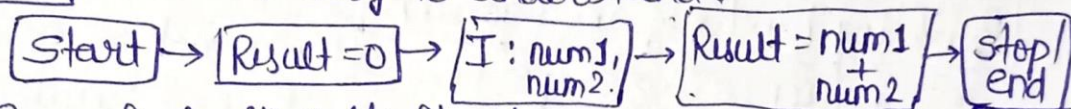
Time Complexity	Type	example
$O(1)$	Constant	add an item to LL (front)
$O(\log n)$	logarithmic	find an item in sorted array (Binary search)
$O(n)$	linear	find an item in unsorted array (Linear search)
$O(n^2)$	Quadratic	nested loop (2 loop)
$O(n^3)$	cubic	matrix multiplication (3x3)
$O(2^n)$	exponential	Fibonacci series (0 1 1 2 3 5.....)

⊕ **Pseudocode** → High level representation of an algorithm (Human understandable)
 → less formal (use of natural language words/elements)
 → No specific syntax (like PL)

Ex Begin

Set result = 0;
 Input : num1, num2
 Result : num1 + num2.
 End.

⊕ **Flowchart** → graphical representation of Algo
 → Diagram to show the flow of data
 → different shapes (□, ◇)
 → easy to understand.



⊕ **Recurrence** → (Recursion) fⁿ calls itself.
 → large problem, divide it → solve it → combine it.

⊗ Recursion is expressed in → "Recurrences"

⊗ **Recurrence eqⁿ** → when fⁿ calls itself its running time is described/given by "Recurrence eqⁿ".

$$T(n) = T(n/2) + C$$

Substitution method :-

$$T(n) = T(n/2) + C \quad n > 1$$

$$1, \quad n = 1.$$

$$\text{substitute } n = n/2$$

$$T(n/2) = T(n/4) + C$$

$$T(n/4) = T(n/8) + C$$

$$\text{Now back substitute,}$$

$$T(n) = T(n/4) + C + C$$

$$= T(n/2^2) + 2C.$$

$$\text{Same for subsequent eqⁿ.}$$

$$= T(n/8) + 2C + C$$

$$= T(n/2^3) + 3C$$

$$T(n/2^k) + Kc$$

$$\text{now we want } T(n/2^k) \text{ as "1"}$$

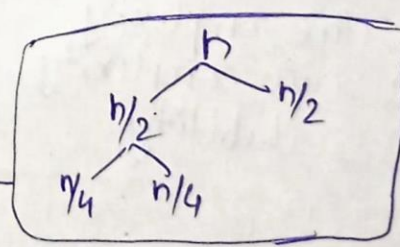
arr → 5 | 10 | 15 | 20 | 25 | 30 | 35

x=5?

5 10 15

25 30 35

5 match.



So, substitute $2^k = n$

$$T(n/2^k) + Kc$$

$$= T(1) + Kc$$

$$\text{for } n=1 \quad K \log_2 = \log n$$

$$T(1)=1 \quad \therefore K = \log n$$

$$\Rightarrow 1 + \log n \cdot C$$

reduce to

$$\Rightarrow O(\log n)$$

infotainer vk

⑧ **Master Theorem/Method** → is a mathematical technique used to analyze & solve recurrence relations that commonly arise in **divide & conquer** algorithms.

→ faster way
→ for few not all Rec. Relation

→ direct way to find time complexity.

$$T(n) = aT(n/b) + f(n)$$

$$a > 1 \quad b > 1$$

n = size of input
 a = no. of subproblems in recursion
 n/b = size of each subproblem
 $f(n)$ = cost of dividing (Problem) & combining (Solution)

Limitations →

- ① $T(n) = \cos n \times$
- ② $f(n) = 2^n$
- ③ $a = 5n$
- ④ $a < 1$

Solution by MM:

$$T(n) = n^{\log_b a} [X] \rightarrow \frac{f(n)}{n^{\log_b a}}$$

$$X = \frac{f(n)}{n^{\log_b a}}$$

① n^p ; $p > 0 \rightarrow O(n^p)$

② n^p ; $p < 0 \rightarrow O(1)$

③ $(\log n)^q$; $q \geq 0 \rightarrow \frac{(\log n)^{q+1}}{q+1}$

infotainer vk

Ex $T(n) = 1T(n/2) + C$
 $a=1 \quad b=2 \quad f(n)=\text{const.}$

$$T(n) = n^{\log_b a} [X]$$

$$n^0 = 1 \quad \frac{C}{n^{\log_2 1}} = \frac{C}{n^0} = C$$

So, $T(n) = 1 \cdot C$ (go for ③)

$$= (\log_2 n)^0 \cdot C$$

$$= \frac{(\log_2 n)^{0+1}}{0+1} \cdot C$$

$$= O(\log_2 n)$$

$$1 = (\log_2 n)^0$$

Ex $T(n) = 8T(n/2) + n$
 $a=8 \quad b=2 \quad f(n)=n$

$$T(n) = n^{\log_2 8} [X]$$

$$n^{\log_2 8} = n^3 \quad \frac{n}{n^{\log_2 8}} = \frac{n}{n^3} = n^{-2}$$

$$= n^3 \cdot O(1)$$

$$= O(n^3)$$

Ex $T(n) = T(n/2) + n^2$
 $a=1 \quad b=2 \quad f(n)=n^2$

$$T(n) = n^{\log_2 1} [X]$$

$$n^0 = 1 \quad \frac{n^2}{n^{\log_2 1}} = \frac{n^2}{1} = n^2$$

$$= 1 \cdot O(n^2)$$

$$= O(n^2)$$

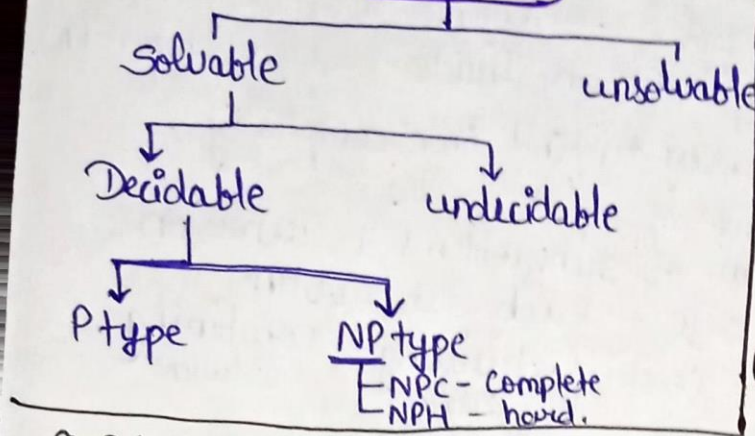
⑨ **Vertex cover problem** → Given an undirected graph $G(V, E)$ and an integer k , determine whether there exists a vertex cover of size $\leq k$.

→ a vertex cover is a set of vertices such that every edge in graph is incident to at least one selected vertex.

→ **class: NP-complete.**

Ex If selecting 3 vertices covers all edges of the graph, then a vertex cover of size 3 exists.

Problem



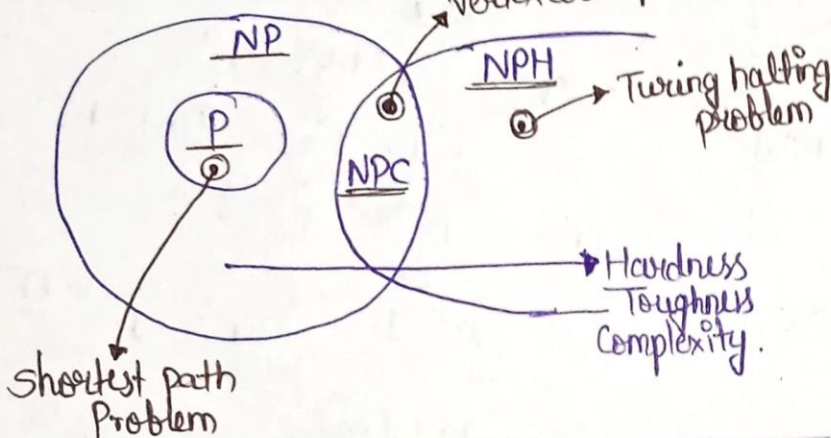
① P type → polynomial type/time
ex → n^k ($n^1, n^2, n^3 \dots$)

① Time complexity = $O(n^k)$

① Efficient & practically solvable

① Both solution & verification are fast

Ex - linear search, binary search



NP-complete

① problem that are in both NP & NP-hard

① exclusively decision problem

① must be NP

① subset of NP-hard

① Polynomial time verification always possible

Ex - CLIQUE, vertex cover, SAT

NP-hard

① Problem that are at least as hard as NP-problem

① Not a decision problem

① Need not to be NP

① Superset of NP-complete

① Polynomial time verification may/maynot be possible.

Ex → Halting problem, optimization TSP

① solvable → can be solved.

① not can be proved that Problem cannot be solved

① unsolvable → can't solve

① can't prove that it cannot be solved

① decidable → Problem that can be solved in finite time or reasonable amount of time

① undecidable → No definite answer for a problem in finite time

① NP-type → Non polynomial type/time.
ex → n^n [$2^n, 3^n \dots$]

① Non deterministic

① verification is fast

① solution may/may not be fast to find

Ex → Hamiltonian path, Travelling Salesman Problem.

① NPC are hardest to solve in NP

① NPC are easiest to solve in NPH

$$P \subseteq NP \subseteq NP\text{-complete}$$

① CLIQUE problem → given an undirected graph $G(V, E)$ and an integer K , determine whether there exists a clique of size K in graph.

→ a clique is subset of vertices such that every pair of vertices is connected by an edge

→ problem asks yes/no (decision)
→ NP-complete.

Ex → If a graph contains a complete subgraph of 4 vertices, then it has a 4-clique.

① Clique can be polynomially reduced to vertex cover.

① They are complementary problems in graph theory.

Linear Search → Simple searching technique, element checked sequentially.

Ex

find $x=1$

5	4	2	1	9
---	---	---	---	---

- ① → 5 4 2 1 9 X
 ② → 5 4 2 1 9 X
 ③ → 5 4 2 1 9 X
 ④ → 5 4 2 1 9 (✓)

① **Best case** → element found at 1st position
 no. of comparison = 1

Time complexity = $T(n) = O(1)$

② **Worst case** → element found at last position
 or not present
 no. of comparison = n

Time complexity = $T(n) = O(n)$

Binary Search → algorithm used on a sorted list.

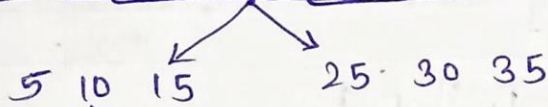
It works by repeatedly dividing the search interval into two halves.

Ex

arr =

5	10	15	20	25	30	35
---	----	----	----	----	----	----

$x=5?$



5 10
 5
 match.

① **Best case** → element found in 1st comparison (middle element)

⇒ no. of comparison = 1

Time complexity = $O(1)$

② **Worst case** → element is found after maximum divisions or not present
 ⇒ The array is divided at 2 at each step.

$n \rightarrow n/2 \rightarrow n/4 \dots$

⇒ No. of comparison = $\log_2 n$

Time complexity = $O(\log n)$

Recursive Binary search →

applies divide & conquer strategy to search an element in a sorted array by repeatedly dividing the array into two halves using recursion.

Recursive relation $T(n) = T(n/2) + c$

Worst case → element not present / found at last level of recursion.

at each recursive call, the problem size is reduced to half.

Time complexity = $O(\log n)$

Space comp. worst case

due to recursion each call uses stack space.

max. recursion depth = $\log n$

space complexity = $O(\log n)$

Q. How many comparisons are needed for a binary search in set of 512 elements?

A. max. no. of comparison = $\log_2 n$ (worst case)

$n = 512$, $512 = 2^9$

$\log_2 512 = 9$

Q. x^{283} left to right binary exponentiation

A. exponent in binary

2	283	
2	141	1
2	70	1
2	35	0
2	17	1
2	8	1
2	4	0
2	2	0
	1	

$283_{10} = 100011011_2$

Start with $y = 1$

Square $y \rightarrow y^2$

If bit = 1, multiply by x

$y = y \cdot x$

Bit	operation	Result
1	$y = 1^2 \cdot x$	x
0	$y = x^2$	x^2
0	$y = (x^2)^2$	x^4
0	$y = (x^4)^2$	x^8
1	$y = (x^8)^2 \cdot x$	x^{17}
1	$y = (x^{17})^2 \cdot x$	x^{35}
0	$y = (x^{35})^2$	x^{70}
1	$y = (x^{70})^2 \cdot x$	x^{141}
1	$y = (x^{141})^2 \cdot x$	x^{283}

Bubble Sort → biggest element on proper position (at last).

Ex →

5	4	2	1	3
---	---	---	---	---

①

5	4	2	1	3
---	---	---	---	---

5 > 4 swap

4	5	2	1	3
---	---	---	---	---

②

4	5	2	1	3
---	---	---	---	---

4	2	5	1	3
---	---	---	---	---

③

4	2	5	1	3
---	---	---	---	---

4	2	1	5	3
---	---	---	---	---

④

4	2	1	5	3
---	---	---	---	---

4	2	1	3	5
---	---	---	---	---

①

4	2	1	3	5
---	---	---	---	---

2	4	1	3	5
---	---	---	---	---

②

2	4	1	3	5
---	---	---	---	---

2	1	4	3	5
---	---	---	---	---

③

2	1	4	3	5
---	---	---	---	---

2	1	3	4	5
---	---	---	---	---

①

2	1	3	4	5
---	---	---	---	---

1	2	3	4	5
---	---	---	---	---

2 < 3 (no swap)

Algo → for i=1 to n
 { K=1
 while (K ≤ n-i)
 { if a[K] > a[K+1]
 { swap a[K], a[K+1]
 K=K+1.
 }
 }

No. of comparison = $\frac{n(n-1)}{2}$ [n = list size]

infotainer vk

Selection Sort → smallest element at position 1.

Ex →

10	6	5	7	1
----	---	---	---	---

①

10	6	5	7	1
----	---	---	---	---

 min=0
②

10	6	5	7	1
----	---	---	---	---

 min=1
③

10	6	5	7	1
----	---	---	---	---

 min=2
④

10	6	5	7	1
----	---	---	---	---

 min=2
⑤

10	6	5	7	1
----	---	---	---	---

 min=4
swap

algo. → SS(arr,).
 { for (i=0; i < n-1; i++)
 { min=i
 for (j=i+1; j < n; j++)
 { if (arr[j] < arr[min])
 min=j
 }
 swap(arr[min], arr[i])
 }
 }

1)

1	6	5	7	10
---	---	---	---	----

 min=1
2)

1	6	5	7	10
---	---	---	---	----

 min=2
3)

1	6	5	7	10
---	---	---	---	----

 min=2
4)

1	6	5	7	10
---	---	---	---	----

 min=2
swap

Insertion sort → Key element Ke left side me save sort pehle

Ex →

7	6	3	4	1
6	7	3	4	1
6	3	7	4	1
3	6	7	4	1

1)

1	5	6	7	10
---	---	---	---	----

 min=2
2)

1	5	6	7	10
---	---	---	---	----

 min=2
3)

1	5	6	7	10
---	---	---	---	----

 min=2
4)

1	5	6	7	10
---	---	---	---	----

 min=3
already sorted.

Final →

1	5	6	7	10
---	---	---	---	----

① **Brute Force method** :- traverse every possible edge/way

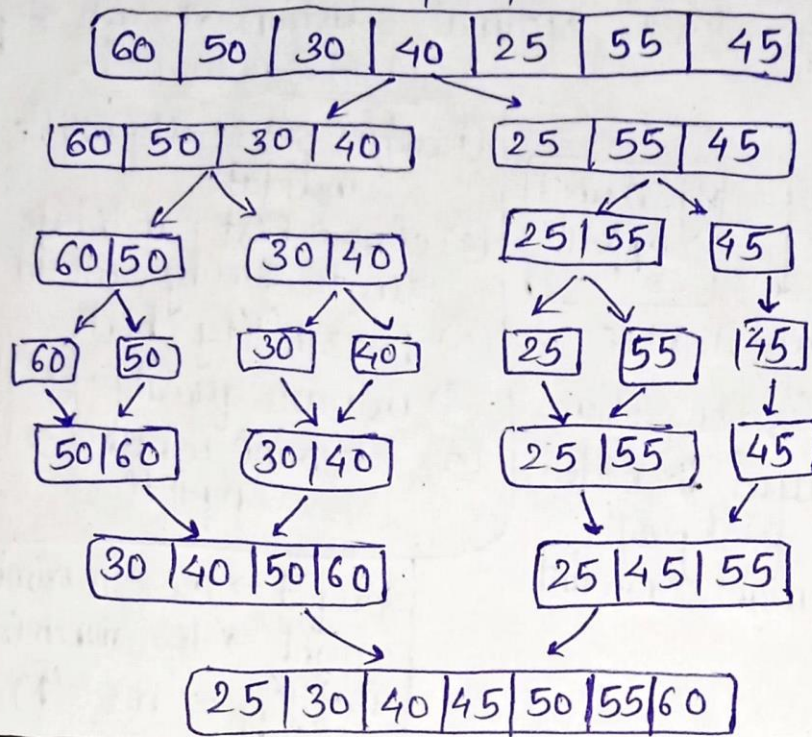
② **Back tracking** → Brute force method + condition

if fail → Backtrack (don't go ahead)
if success → go ahead.

Ex → vehicle can't go through holes/slippy road



Merge Sort → divide & conquer sorting algorithm.
 → splits into 2 till last and then merges them back



Algo. → Merge Sort(start, end).
 { if (p < r)
 { m = (p+r)/2
 mergeSort(arr, p, m)
 mergeSort(arr, m+1, r)
 merge(arr, p, m, r).
 }

Recurrence relation $T(n) = 2T(\frac{n}{2}) + n$

Time complexity $T(n) = O(n \log n)$
 (Both best & worst)

space complexity = $O(n)$

Quick sort → divide & conquer algo.
 → pivot element → 2 sublist (element smaller & element greater)

ex → [15 | 10 | 13 | 9 | 12 | 7]

pivot = 7

7 | 10 13 9 12 15

pivot = 15

7 | 10 13 9 12 | 15

pivot = 12

7 | 10 9 | 12 13 15

pivot = 9

7 | 9 | 10 | 12 | 13 | 15

(Sorted)

Algo → Partition(arr, p, r).
 { pivot = arr[r]
 i = p - 1
 for j = p to (r-1)
 { if (arr[j] ≤ x)
 { i = i + 1
 swap(arr[i], arr[j])
 }
 return i + 1;
 }

QS(arr, p, r).
 { if (p < r)
 { x = partition(arr, p, r)
 QS(arr, p, x-1)
 QS(arr, x+1, r)
 }

Strassen's Algorithm → normal multiply (2x2) → 8 $O(n^3)$
 strassen → 7

$T(n) = O(n^{2.81})$

ex → $\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} e & f \\ g & h \end{bmatrix} = \begin{bmatrix} ae+bg & af+bh \\ ce+dg & cf+dh \end{bmatrix}$

$P_1 = a(f-h)$

$P_2 = (a+b)h$

$P_3 = (c+d)e$

$P_4 = d(g-e)$

$P_5 = (a+d)(e+h)$

$P_6 = (b-d)(g+h)$

$P_7 = (a-c)(e+f)$

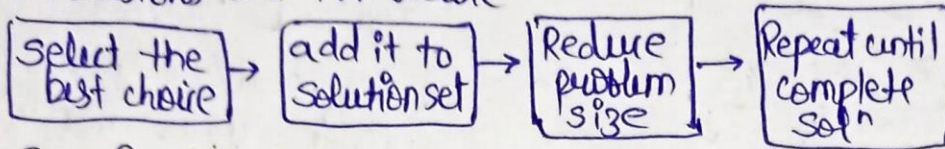
$$= \begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{bmatrix}$$

$T(n) = 8T(n/2) + n^2 \rightarrow O(n^3)$

Strassen $T(n) = 7T(n/2) + n^2 \rightarrow O(n^{2.81})$

infotainer vk

Greedy approach → solve the problem with best optimal solution out of all feasible solutions.
 → it does not reconsider previous choices
 → Look for local optimal solution at each step
 → decisions are irrevocable



Ex → Fractional Knapsack, Prim, Kruskal MST

Knapsack Problem → optimization problem
 → Fractional → fraction allowed in order to maximize total profit. Subject to weight constraint.
 ex-biscuit.
 → 0/1 → All or none.
 ex-laptop.

Q. no. of object = 7, Knapsack capacity = 5
 $W = (2, 3, 5, 7, 1, 4, 1)$
 $P = (10, 5, 15, 7, 6, 18, 3)$

A. compute $P/W = (5, 1.67, 3, 1, 6, 4.5, 3)$

sort in decreasing = $\frac{P}{W} = (6, 5, 4.5, 3, 3, 1.67, 1)$
 $P = (6, 10, 18, 15, 3, 5, 7)$

Capacity = 5 → 1 (full) - 6 profit
 2 (full) - 10 profit
 2/4 (fraction) $18 \times \frac{2}{4} = 9$ profit

Knapsack full now.

Total profit = $6 + 10 + 9 = 25$

Task scheduling Algorithm →

(Job sequencing with deadlines)
 is greedy algorithm used to schedule jobs in such a way that total profit is maximized subject to constraints that:

- ① each job takes one unit of time
- ① each job has a deadline
- ① only one job can be scheduled at a time
- ① A job must be completed on or before its deadline to earn profit.

Activities performed:-

- ① define set of all possible candidate
- ② choose best candidate acc. to greedy criteria
- ③ Feasibility check
- ④ measure quality of soln
- ⑤ determine whether soln is complete

Profit → more/maximize
 weight → less/minimize
 $\frac{\text{Profit}}{\text{weight}} = \text{more}(\uparrow)$

Time complexity = $O(n \log n)$

space complexity = $O(n)$

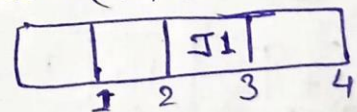
Job sequencing

→ focus on 'Profit'
 ⊕ execute it late.

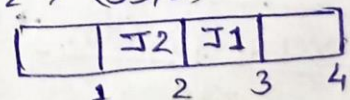
Ex

Jobs	J1	J2	J3	J4	J5	J6
P:	40	35	30	25	20	15
d:	3	3	4	2	1	3

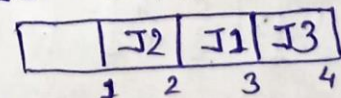
J1 → (40, 3)



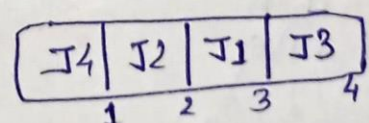
J2 → (35, 3)



J3 → (30, 4)



J4 → (25, 2)



Profit = $40 + 35 + 30 + 25$
 = 130

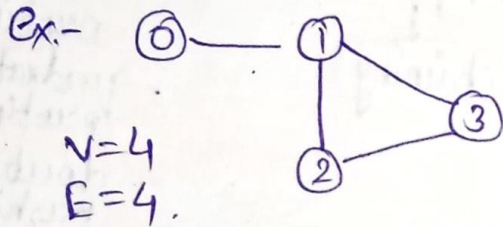
Graph → A non linear data structure used to represent relationships between objects

$$G = (V, E)$$

↓ ↓
vertices (node) edges

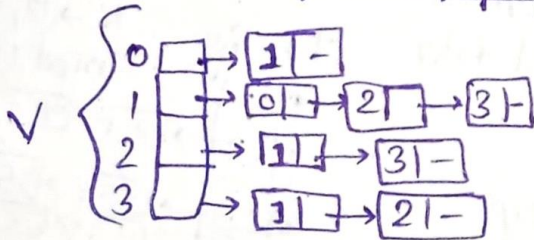
Representation of graph :

- adjacency matrix
- adjacency list



4x4

	0	1	2	3
0	0	1		
1	1	0	1	1
2		1	0	1
3		1	1	0

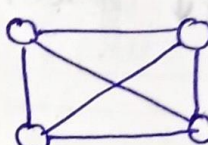


① directed graph $1 \rightarrow 2$

② undirected graph $1 \text{ --- } 2$

③ weighted graph $1 \xrightarrow{5 \text{ w/c}} 2$

④ unweighted graph $1 \text{ --- } 2$

⑤ complete graph 

⑥ simple graph 

infotainer vk

Depth first search (DFS) → graph traversal technique in which we start from a source node and explore as far as possible along one branch before backtracking

Breadth first search (BFS) → graph traversal method in which we explore all nodes level by level starting from source.

Proof by Mathematical Induction

Base case ① $n = 1$ check if LHS = RHS

② $n = K$ (K is any +ve integer)

③ $n = K+1$

Ex.:- $1 + 2 + 3 + 4 + \dots + n = \frac{n \cdot (n+1)}{2}$

① LHS $n = 1$, RHS = $\frac{1(1+1)}{2} = 1$ (True)

② Assume true for $n = K$.

$$1 + 2 + 3 + \dots + K = \frac{K(K+1)}{2}$$

③ $n = K+1$

$$1 + 2 + 3 + \dots + (K+1) = \frac{(K+1) \cdot (K+1+1)}{2}$$

$$\downarrow$$
$$1 + 2 + 3 + \dots + K + (K+1) = \frac{(K+1)(K+2)}{2}$$

$$\downarrow$$
$$\frac{K(K+1)}{2} + (K+1) = \frac{(K+1)(K+2)}{2}$$

$$\frac{(K+1)(K+2)}{2} = \frac{(K+1)(K+2)}{2}$$

LHS = RHS

True for $n = K+1$ ✓

Spanning tree → A connected subgraph 'S' of graph $G(V, E)$ is said to be Spanning if -

- ① 'S' should contain all vertices of 'G'
- ② 'S' should contain $(|V|-1)$ edges.

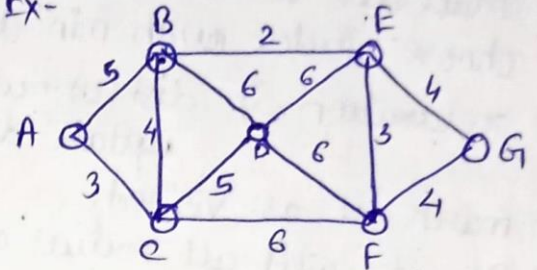
Kruskal Algo. for Minimum cost spanning tree (MCST)

→ it is a greedy algorithm used to find MST of connected, undirected weighted graph.

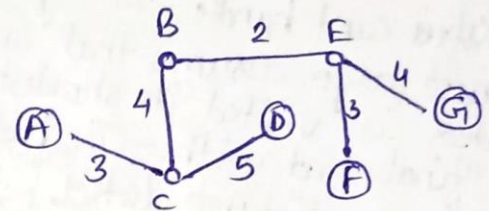
① Steps:-

- ① List all edges of graph with weight
- ② Sort in ascending order of weight
- ③ Initialize MST as empty set
- ④ Pick smallest-weight edge.
 - ① Add it to MST if it does not form cycle
 - ② If cycle forms → skip
- ⑤ Repeat step 4 until MST contain exactly $(V-1)$ edges (V =vertices)

Ex-



$$V-1 = 7-1 = 6$$



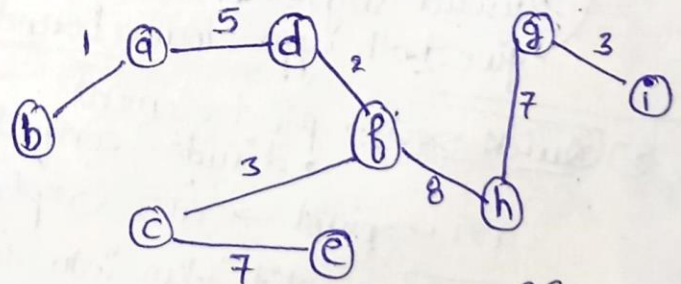
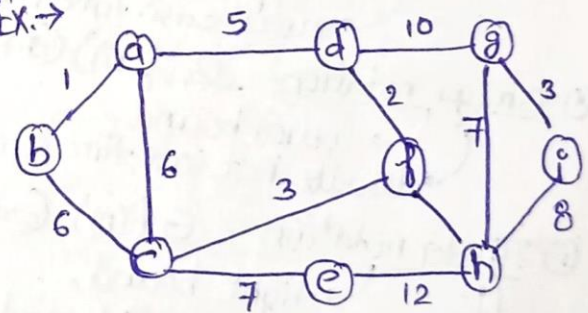
Prim's Algo. for MCST (No loop should be formed) total cost = 21 (sum).

→ is a greedy algorithm that builds the MCST by starting from any one vertex and repeatedly adding minimum-weight edge that connects a selected vertex to an unselected vertex.

• Steps →

- ① Start from any vertex
- ② Mark this vertex as part of MST set
- ③ From all edges that connect MST set to vertices not in MST, choose the edge with smallest weight.
- ④ Add the chosen edge and new vertex to MST
- ⑤ Repeat steps 3-4 until all vertices are included.
- ⑥ Set of selected edges forms MCST

Ex. →



$$\text{Cost} = 36$$

infotainer vk

<u>Prim</u>	<u>Kruskal</u>
① vertex based	① edge based.
① for dense graphs	① for sparse graph
① DS - Priority queue	① DS - unionfind
① start - any vertex	① start - small edge
① Edge sorting not req.	① Edge sorting req.

⊕ Dijkstra's Algorithm → single source shortest path → use greedy approach

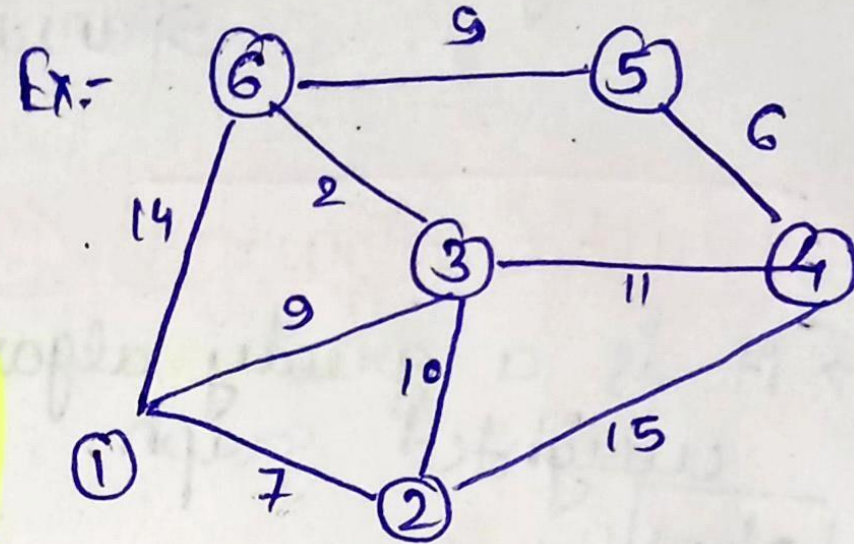
real life ex - google maps.
- graph

Steps :-

- ① Initialize distance of all vertices to ∞ except source
- ② Mark all vertices unvisited
- ③ choose vertex with min. dist. from unvisited set
- ④ relaxation if $\text{dist}(u) + \text{weight}(u, v) < \text{dist}(v)$.
update $\text{dist}(v)$
- ⑤ mark u as visited.
- ⑥ Repeat until all vertices are visited.

* Dijkstra can't handle -ve edge weights because algo. assumes that once the vertex is visited its shortest distance is final but with -ve, path can reduce distance later.

For -ve weight → Bellman-ford Algo.



source	destination				
1	2	3	4	5	6
	∞	∞	∞	∞	∞
1	7	9	∞	∞	14
1,2	7	9	22	∞	14
1,2,3	7	9	20	∞	11
1,2,3,6	7	9	20	20	11
Any → 1,2,3,6,4,5	7	9	20	20	11

infotainer vk

biggest element

Huffman Coding → is a greedy algorithm used for lossless data compression.
 It assigns variable-length binary codes to characters such that:
 ① frequently occurring characters get shorter codes
 ② Rare characters get longer codes.
No code is a prefix of another.

This minimizes the total no. of bits required to represent data.

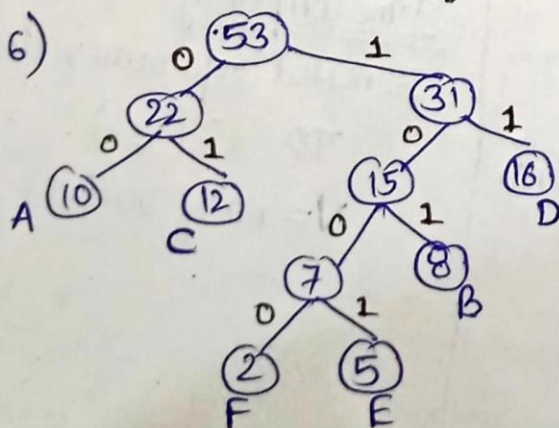
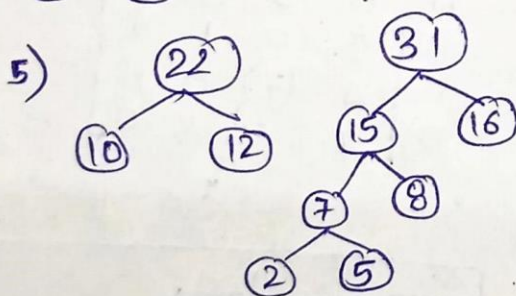
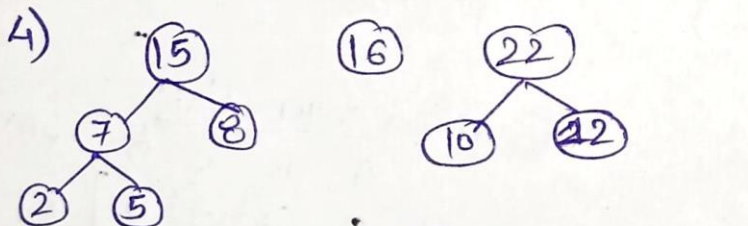
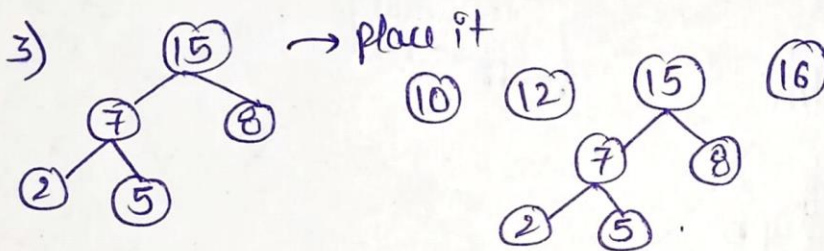
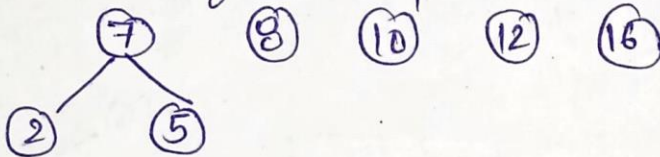
Ex {00, 01, 11, 10} ✓
 {00, 001, 110, 1100} ✗

Ex A B C D E F
 10 8 12 16 5 2

1) ascending order.

2 5 8 10 12 16

2) combine first 2 & place it in order



code:

A: 00

B: 101

C: 01

D: 11

E: 1001

F: 1000

weight = $10 \times 2 + 8 \times 3 + 12 \times 2 + 16 \times 2 + 5 \times 4 + 2 \times 4$
 = 128

Bellman-Ford Algorithm

→ is a single source shortest path algorithm used to find the shortest distances from a given source vertex to all other vertices in a weighted graph, even when the graph contains negative edge weights.

⊛ It can also detect negative weight cycles, which dijkstra's algorithm cannot.

- based on dynamic programming / relaxation
- slower than dijkstra but more robust.

Sum of Subsets problem

Given set = {1, 3, 5, 9}

Sum = 8

operation output → T/F Yes/No

	1	2	3	4	5	6	7	8
1	T	F	F	F	F	F	F	F
3	T	F	T	T	F	F	F	T
5	T	F	T	T	T	T	F	T
9	T	F	T	T	T	T	F	T

{1}, {1,3}, {1,3,5}, {1,3,5,9}

Dynamic programming approach → is a problem solving technique used to solve optimization and decision problems by breaking them into overlapping subproblems, solving each subproblem only once, & storing their results to avoid repeated computation.

applicable when:

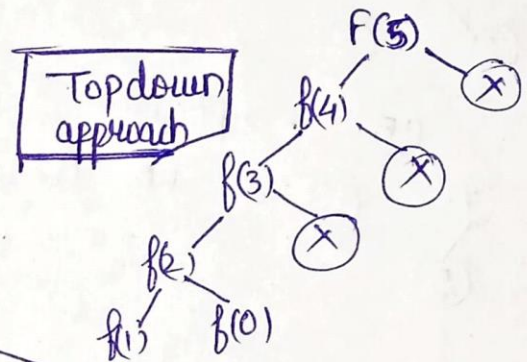
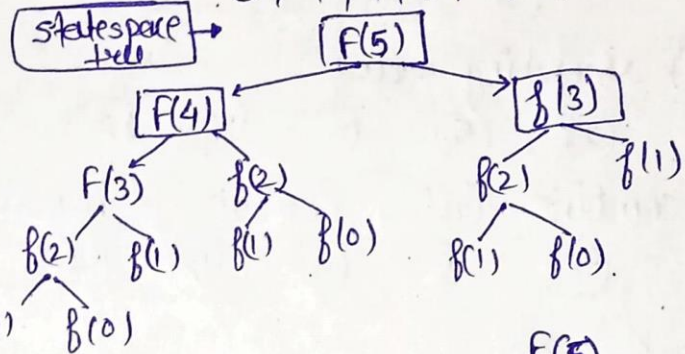
- ① Optimal substructure
- ① Overlapping subproblems

- uses recursion + memory
- ensures global optimal solution
- converts exponential time solⁿ → polynomial time.

Steps →

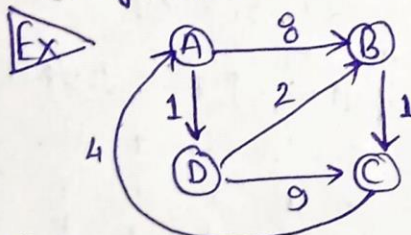
- ① Characterize the problem
- ② Define subproblems
- ③ Form recurrence relation
- ④ Compute subproblems
- ⑤ Store results
- ⑥ Construct final solution.

EX ▶ Fibonacci sequence.
 $f(0) = 0, f(1) = 1, f(n) = f(n-1) + f(n-2)$
 0, 1, 1, 2, 3, 5, 8.



Floyd Warshall Problem :-

- All pair shortest path
- every vertex to every other vertex shortest distance.
- is a dynamic programming algorithm used to find the shortest path distances between every pair of vertices in a directed weighted graph (may contain -ve weights but no -ve cycles)



If have self loops / parallel edges remove them.

① Direct edge

	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	∞	0	∞
D	∞	2	9	0

② 'A' as intermediate

	A	B	C	D
A	0	8	∞	1
B	∞	0	1	∞
C	4	12	0	5
D	∞	2	9	0

③ 'B' as intermediate

	A	B	C	D
A	0	8	9	1
B	∞	0	1	∞
C	4	12	0	5
D	∞	2	3	0

④ 'C' as intermediate

	A	B	C	D
A	0	8	9	1
B	5	0	1	6
C	4	12	0	5
D	7	2	3	0

→ BCAD
 → DBCA

⑤ 'D' as intermediate

	A	B	C	D
A	0	3	4	1
B	5	0	1	6
C	4	7	0	5
D	7	2	3	0

→ CADB

Time complexity

3 nested loops over vertices

$O(V^3)$

V = no. of vertices

infotainer vk

Polynomial Evaluation is process of computing the value of a polynomial for a given value of variable.

Methods:-

① Direct method (Naive)

- compute each power of x separately
- Time complexity: $O(n^2)$

② Horner's Rule

- Rewrites polynomial to reduce multiplications
- Time complexity: $O(n)$
- most efficient.

Ex → By Horner's rule.

$$P(x) = 3x^2 + 5x + 6$$
$$(x=3)$$

Horner:-

$$P(x) = (3x + 5)x + 6$$

$$x=3 \rightarrow 3 \times 3 + 5 = 14$$

$$14 \times 3 + 6 = 48$$

$$P(3) = 48$$

Euclid's Algorithm for GCD

Problem → find Greatest common divisor of 2 natural no. m & n .

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

process is repeated until remainder becomes 0

The last non-zero remainder is GCD

infotainer vk

Algo

Euclid GCD(m, n)

while $n \neq 0$ do

$r = m \bmod n$

$m = n$

$n = r$

end while

return m

Time complexity = $O(\log \min(m, n))$

Matrix chain Multiplication

is a dynamic programming problem that determines the most efficient way (optimal parenthesization) to multiply a sequence of matrices so that the no. of scalar multiplications is minimized

Note: order does not change, only parenthesization change

$A_1: 30 \times 35$

$A_2: 35 \times 15$

$A_3: 15 \times 5$

dimension array $p = [30, 35, 15, 5]$

possible parenthesization

① $(A_1 A_2) A_3$

$$A_1 A_2: 30 \times 35 \times 15 = 15750$$

$$(A_1 A_2) A_3: 30 \times 15 \times 5 = 2250$$

$$\text{Total} = 18000$$

② $A_1 (A_2 A_3)$

$$A_2 A_3: 35 \times 15 \times 5 = 2625$$

$$A_1 (A_2 A_3): 30 \times 35 \times 5 = 5250$$

$$\text{Total} = 7875$$

(minimum)

String matching Algorithm

is used to find all occurrences of a pattern string P of length m in a text string T of length n .

Naive (Brute force)

① align the pattern P with text T starting from left

② compare characters one by one

③ If a mismatch occurs, shift the pattern by one position & repeat

④ continue until the pattern is found or the text ends.

Best case $T(n) = O(n)$

only one comparison

Final answer → $A_1 \times (A_2 \times A_3)$

Tractable problem

- ① problem that can be solved in polynomial time
- ① Belongs to class P
- ① considered efficiently solvable
- ① Ex - sorting, Bellmanford, Dijkstra

Intractable problem

- ① Problems that can't be solved in polynomial time.
- ① Belongs to NP-Hard NP-complete.
- ① require exponential time in worst case
- ① Ex - TSP, Knapsack (0/1) clique

① Optimization problem

aim to find the best solution.
ex - shortest path
max. profit knapsack.

① Decision problem

answer is Yes/No
- used in complexity theory.
ex → does a clique of size K exist?

optimization problem can be converted into decision problem.

Approximation Algorithms → is a polynomial

why?

- exact solutions are computationally infeasible
- used when efficiency is more important than optimality.

time algorithm that produces a solution close to the optimal solution for NP-Hard optimization problems.

Ex → approximation of TSP

Cook - Levin Theorem / CNF-Satisfiability Theorem

→ It states that Boolean Satisfiability problem (SAT) - specifically CNF-SAT is NP-complete.

CNF-SAT $\in$ NP & every problem in NP can be reduced to CNF-SAT in polynomial time.

Greedy Approach

- ① Makes locally optimal choice at each step
- ① decisions are irrevocable
- ① optimality not always guaranteed
- ① Simple & fast
- ① Time complexity ($\downarrow$) low
- ① Ex → Fractional Knapsack, Prim, Kruskal

Dynamic approach

- ① Solves problem by combining solutions of subproblems
- ① decisions can be revised
- ① Always guarantees optimal solution
- ① More complex & systematic
- ① Time complexity high than greedy
- ① Ex → 0/1 Knapsack

infotainer vk

Thank you

Like the video & subscribe the channel